Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Session-Management

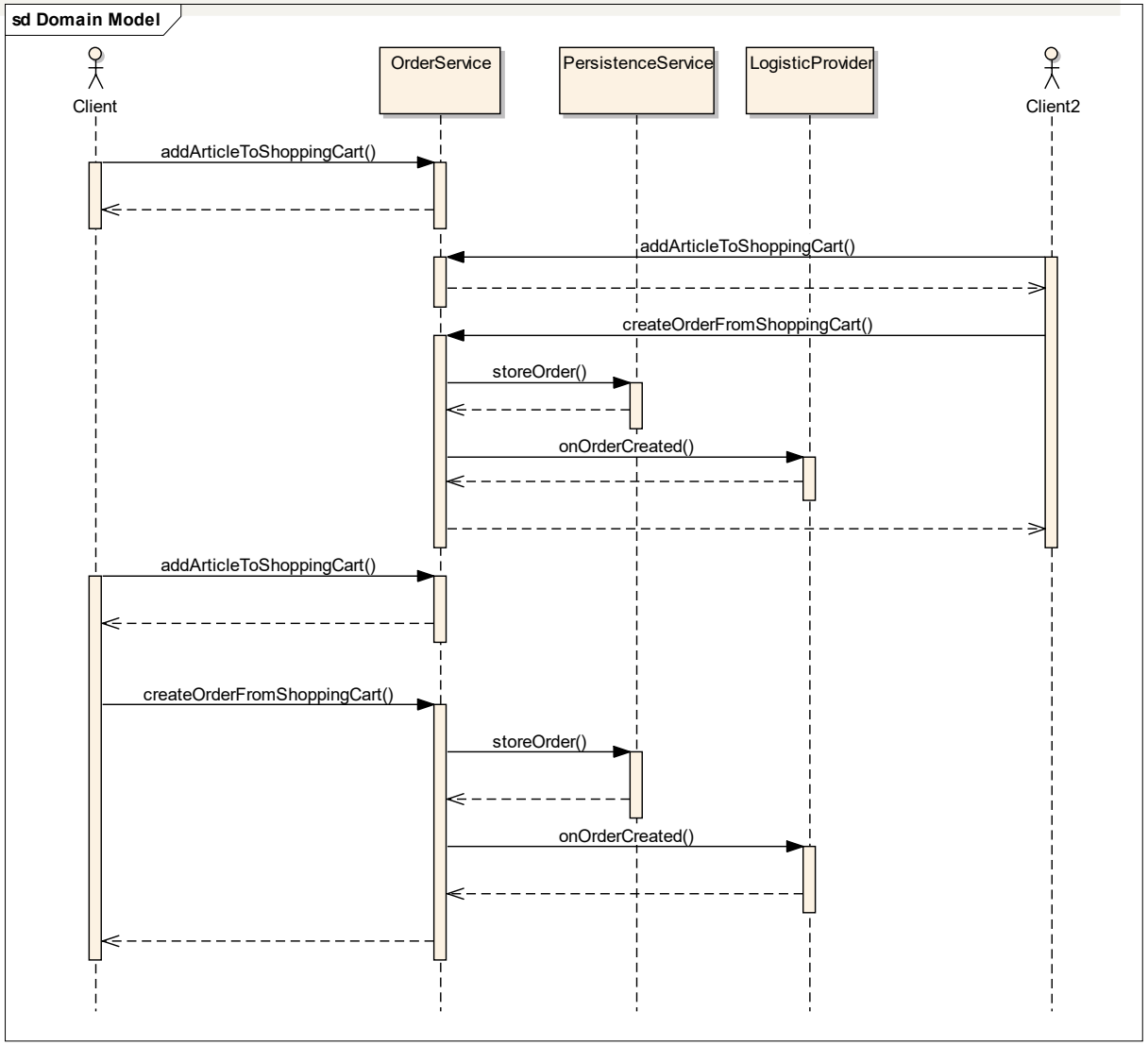
Dokumenthistorie

Autor:	Prof. Dr. Bernd Hafenrichter
Version:	1.1
Letze Änderung:	16.03.2025

Ausgangssituation

- Es soll eine E-Commerce-Applikation (HTTP) entwickelt werden, welche eine Warenkorb-Funktionalität anbietet.
- Der Warenkorb wird server-seitig verwaltet
- Ein Benutzer kann Artikel schrittweise in den Warenkorb legen
- Die Implementierung soll möglichst einfach gehalten sein

Ausgangssituation



Konzeptionelle Serverpatterns – Session Context Manager

Problem

- Wie kann man ein verbindungsorientiertes Protokoll implementieren, welches über mehrere Aufrufe hinweg einen gemeinsamen Kontext verwaltet?
- Wie kann ein Worker Multiple Request Sessions bearbeiten, ohne exklusiv einem Client zugeordnet zu sein?
- Ist ein Worker genau einem Client für dessen Session zugeordnet, verbringt er einen größeren Teil der Zeit mit Warten auf den nächsten Service Request und blockiert damit Ressourcen.
- Reißt die Verbindung ab oder beendet sich die Worker Task wegen eines Fehlers, sind auch die Sitzungsdaten verloren.

Konzeptionelle Serverpatterns – Session Context Manager

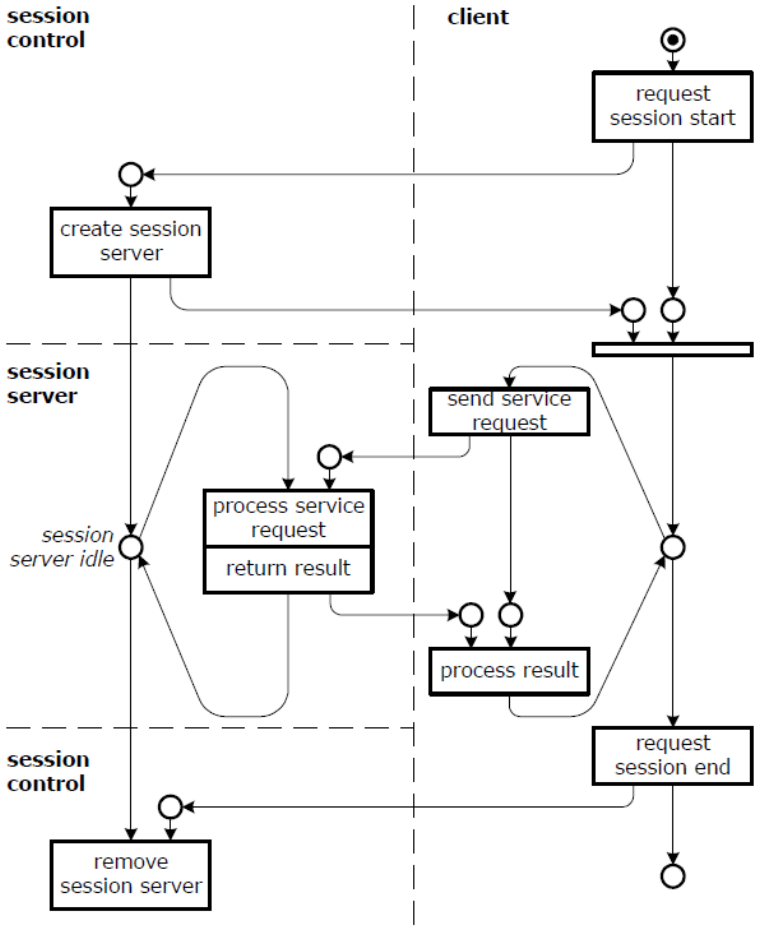
Pattern: Session Context Manager

Lösung:

- Man wendet für einen auftragsbearbeitenden Server das LISTENER / WORKER Pattern an und möchte Multiple Request Sessions handhaben, also den Kontext zwischen mehreren Requests eines Clients erhalten.
- Ein Session Context Manager verwaltet Zustand und Daten einer Multiple Request Session
- Ein Worker bearbeitet immer genau einen Service Request, indem er den zugehörigen Kontext vom Manager anfordert, den Service Request beantwortet und den geänderten Kontext an den Manager zurück gibt.
- Zur Identifikation der Session wird eine ID verwendet
- Der Client muss die ID zusammen mit dem Service Request übertragen

Konzeptionelle Serverpatterns

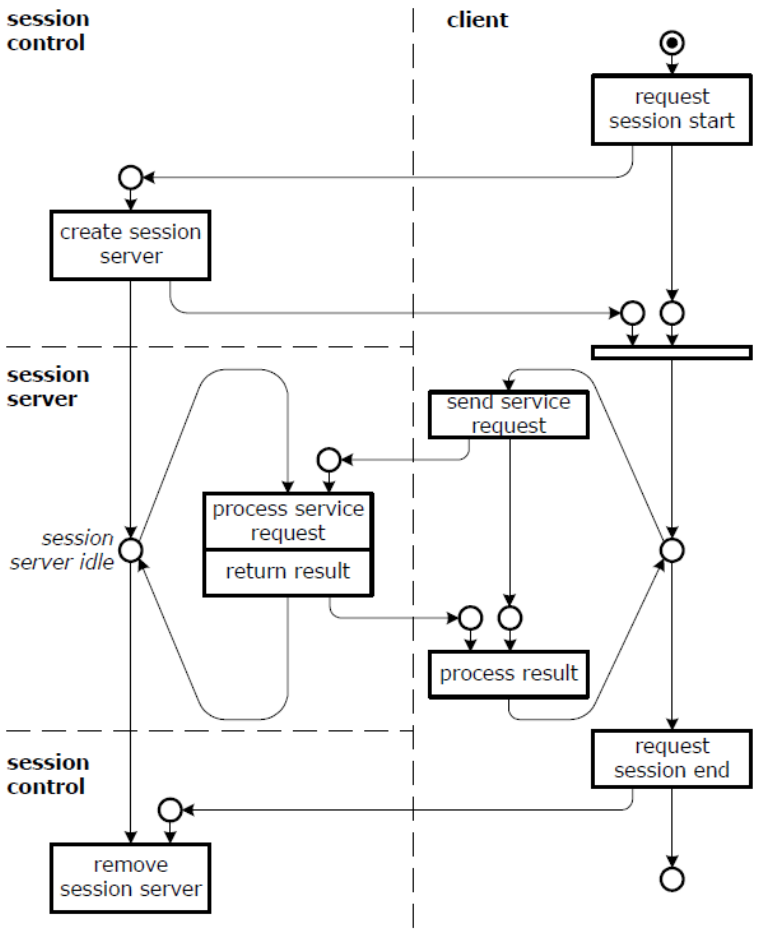
Keine Verwechslung mit TCP/IP



Multi-Request-Session:

1. Der Client baut eine Sitzung mit dem Server auf
2. Der Client sendet den Auftrag an den Server
3. Der Server verarbeitet den Auftrag
4. Der Server sendet die Antwort zurück an den Client
5. Die Schritte 2 – 4 werden wiederholt durchgeführt
6. Die Verbindung zwischen Client uns Server wird wieder abgebaut

Konzeptionelle Serverpatterns

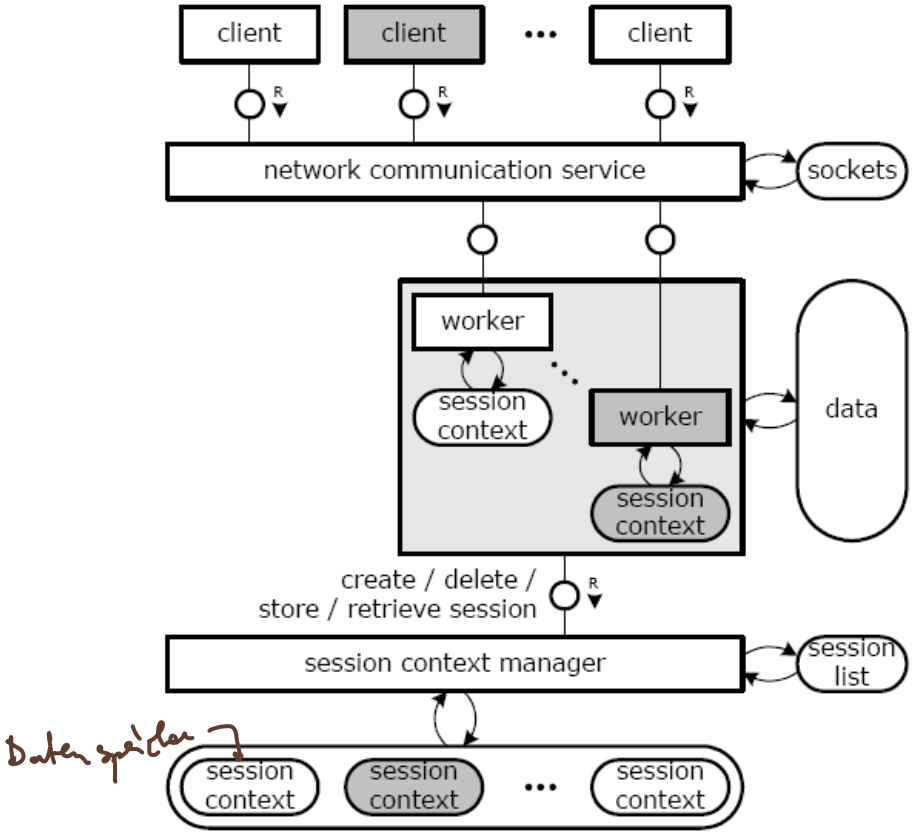


Folgende Zeiten sind relevant für den Client:

- Connect Time: Die Dauer zwischen Senden einer (TCP-) Verbindungsanfrage und Aufbau der Verbindung
- Response Time-1: Zeit für die Antwort auf den ersten Auftrag nach Verbindungsaufbau
- Response Time-2: Zeit für die Antwort auf weitere Aufträge bei bestehender Verbindung

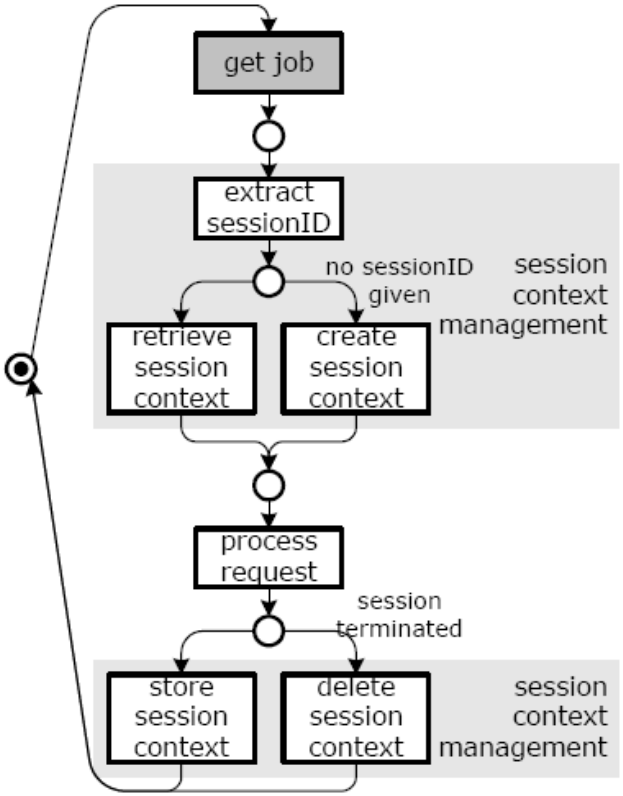
Konzeptionelle Serverpatterns – Session Context Manager

Statische Sichtweise



Konzeptionelle Serverpatterns – Session Context Manager

Dynamische Sichtweise



Worker Pool pattern benutzen?

Konzeptionelle Serverpatterns – Session Context Manager

Eigenschaften:

- Die Worker Tasks werden optimal ausgenutzt, da sie nicht zwischen zwei Service Requests eines Clients für andere Requests blockiert sind.
- Da ein Client seine Session über eine ID identifizieren muss, ist im Fall eines Verbindungsabbruchs oder im Fehlerfall einer Worker Task das Wiederherstellen einer Session möglich.

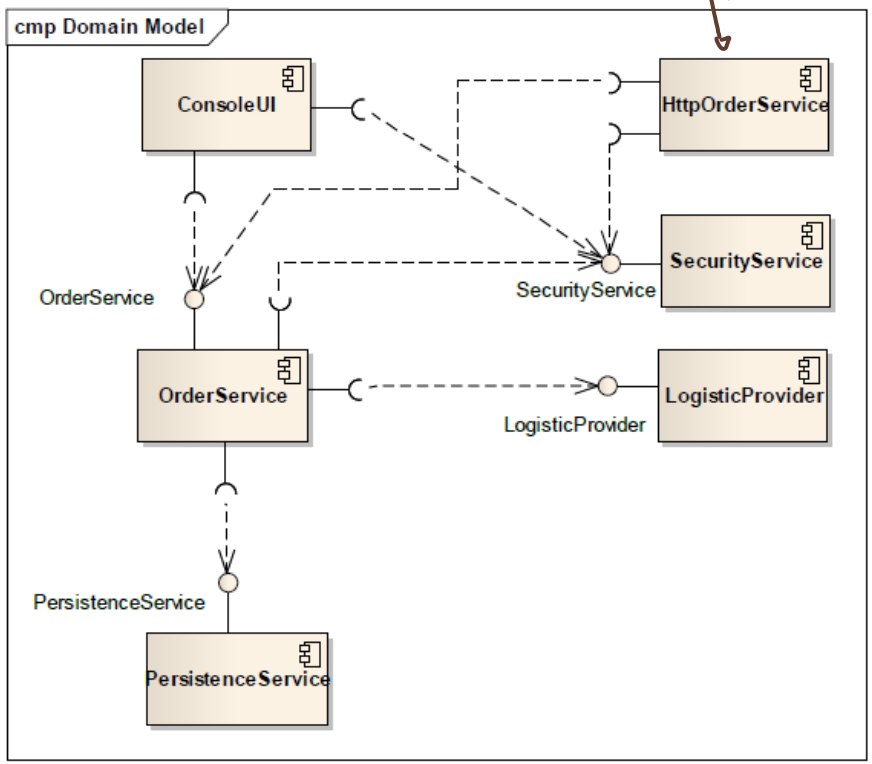
Konzeptionelle Serverpatterns – Session Context Manager

Offene Fragen/Punkte

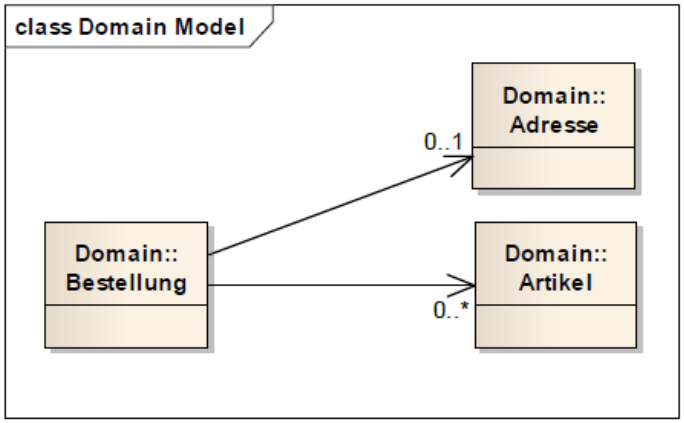
- Ein Client muss eine Session ID speichern und mit jedem Service Request mitschicken
- Man muss damit rechnen, dass eine Session nicht sauber beendet wird. Das erfordert eine Garbage Collection im Session Context Speicher.
- Die Antwortzeit innerhalb einer Session erhöht sich, da bei jedem Service Request erst der Session Context angefordert werden muss.

Architektur der Beispielapplikation

R-Software

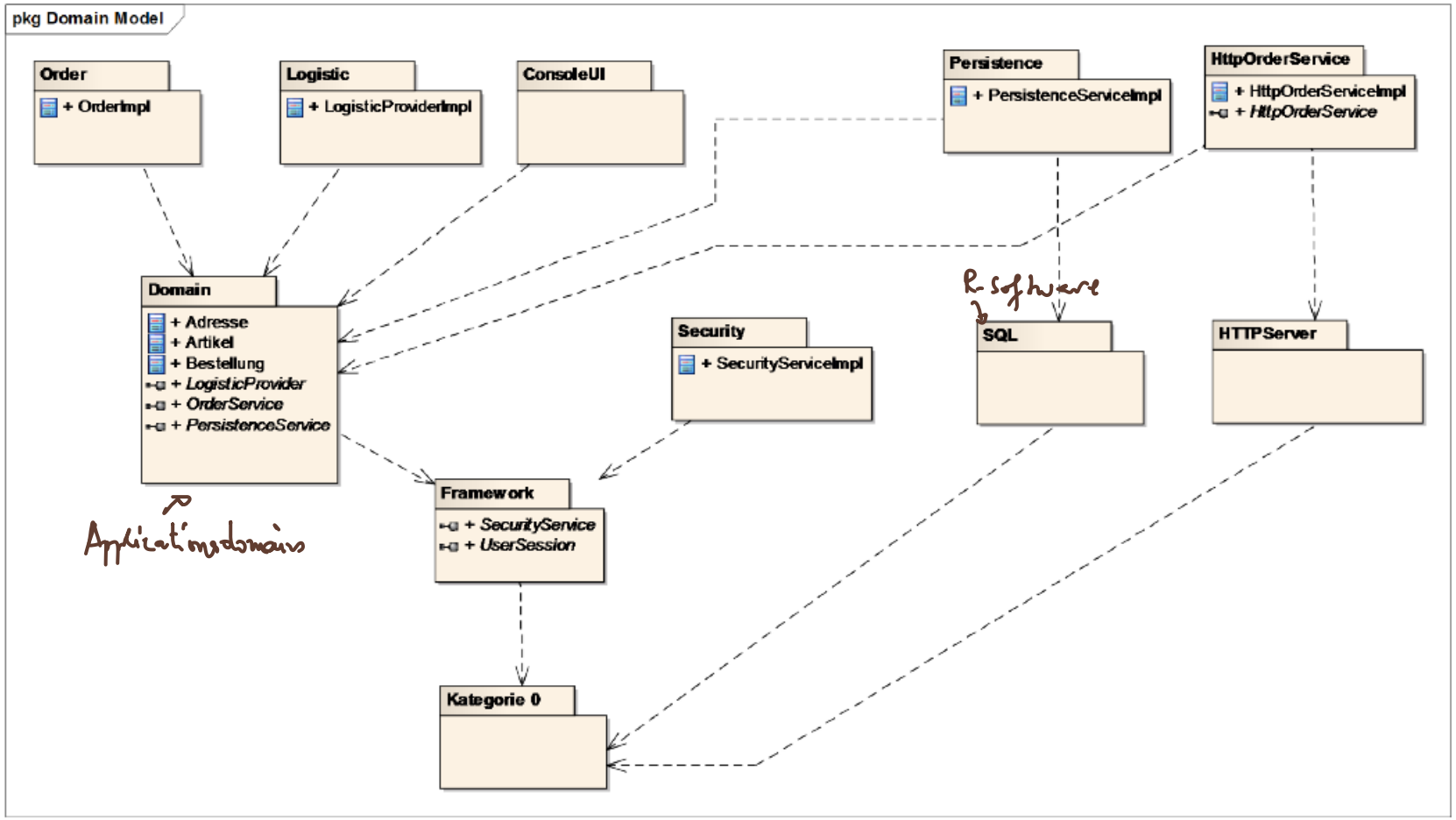


Datenmodell



Prüfung & Security wie möglich verankert?

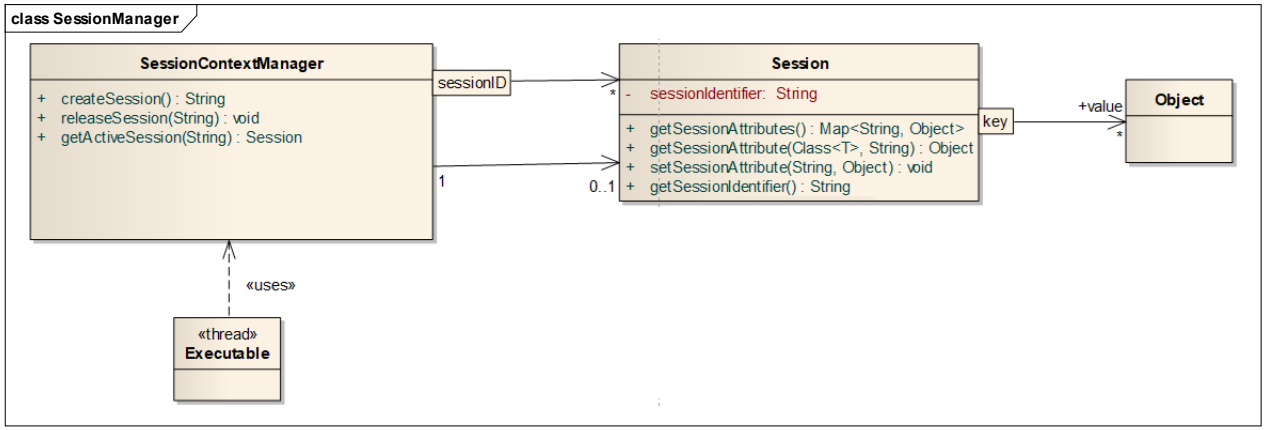
Der Kategoriegraph der Beispielapplikation



Lösung 1 – Explizites Session Management

Lösungsidee:

- Ein SessionContextManager verwaltet eine beliebige Anzahl von Session
- Innerhalb einer Session können beliebige Attribute gespeichert werden
- Alle Komponenten bekommen explizit ein Session-Objekt über die Schnittstelle übergeben.
- Der Client selber ist dafür verantwortlich die Session an alle notwendigen Stellen zu übergeben



Lösung 1 – Explizites Session Management

```
public interface OrderService {

/**
 * Add a new article to the currently open order (
*/

public void addArticleToShoppingCart( Session s, Artikel a );

/**
 * Reset the shopping cart to initial state
*/

public void resetShoppingCart( Session s );

/**
 * Add a new article to the currently open order (
*/

public List<Artikel> getShoppingCart( Session s );

/**
 * Create a new order from the currently active shopping cart
*/

public String createOrderFromShoppingCart( Session session, Adresse a ) throws DomainException,
SecurityServiceException;
}
```

Explizite Übergabe der Session an die Komponenten

Warum keine globalen Variablen?

Lösung 2 – Implizites Session Management mit threadlokalen Variablen

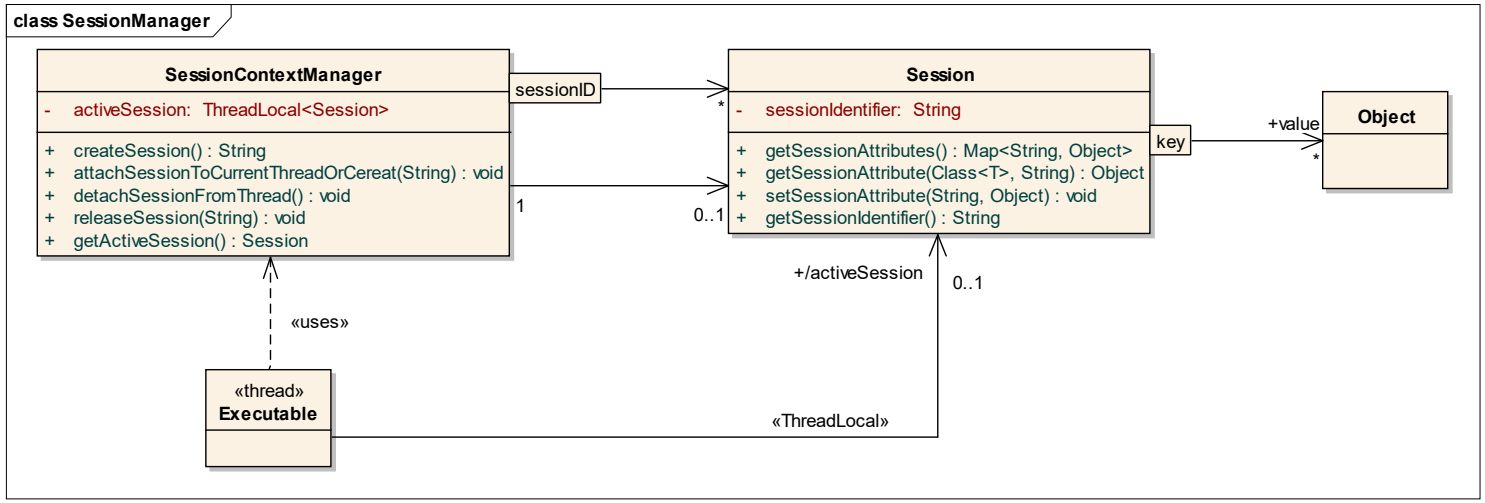
Problem von Lösung 1:

- Ein Architekt muss zur Design-Zeit vorhersehen welche Komponenten alle Session-Information benötigen
- Das Argument „Session“ wirkt unnatürlich in einem Komponenten-Interface da es nicht fachlich motiviert ist.
- Komponenten-Schnittstellen werden überfrachtet mit unnötigen Informationen

Lösunsgsidee:

- Implizites Session-Management mit Hilfe von thread-lokalen Variablen
- Eine Session ist mit dem aktuellen Thread (per threadlokale Variable) verknüpft.
- Keine explizite Übergabe ist nicht notwendig

Lösung 2 – Implizites Session Management mit threadlokalen Variablen



Lösung 2 – Implizites Session Management mit threadlokalen Variablen

```
public interface OrderService {

/**
 * Add a new article to the currently open order (
 * @param a
 */

public void addArticleToShoppingCart( Artikel a );

/**
 * Reset the shopping cart to initial state
 */

public void resetShoppingCart();

/**
 * Add a new article to the currently open order (
 */

public List<Artikel> getShoppingCart();

/**
 * Create a new order from the currently active shopping cart
*/

public String createOrderFromShoppingCart( Adresse a ) throws DomainException,
SecurityServiceException;
}
```

- Keine Übergabe der Session an die Komponenten notwendig
- Schnittstelle ist „rein“ und technologieneutral

Lösung 2 – Implizites Session Management mit threadlokalen Variablen

```
public class OrderServiceImpl {

      /**
       * Add a new article to the currently open order (
       * @param a
       */

      public void addArticleToShoppingCart( Artikel a ) {

            ShoppingCart cart = SessionContextManager.getActiveSession().getSessionAttribute(
                                                      ShoppingCart.class, "ShoppingCart" );

            if( cart == null ) {
                  cart = new ShoppingCart();
                  SessionContextManager.getActiveSession().setSessionAttribute( "ShoppingCart",
                                                                                cart );
            }

            cart.addArticle( a );
      }
```

- Session-Handling wird in die Komponente verlagert

Lösung 3 – Implizites Session Management mit Scopes

Problem von Lösung 2:

- Das Session Handling wird dem Entwickler einer Komponente überlassen.
- Redundanter Code beim Zugriff auf die Session
- Unnötige Komplexität und erhöhtes Fehleraufkommen

- **Lösunsgsidee:**
  - Kann die Komponentenimplementierung wie in einer „Single-User-Umgebung“ erfolgen?
  - Komplexität des Session-Handling wird in das Framework verlagert
  - Session-Abhängige Objekte werden als Komponente definiert
  - Das Framework sorgt dafür das unterschiedliche Instanzen (pro Session) erzeugt werden.

Lösung 3 – Implizites Session Management mit Scopes

```
@ManagedComponent(      name        = "ShoppingCart",
                        description = "The implementation of the shopping cart",
                        scope       = Scope.Session )

public class ShoppingCartImpl implements ShoppingCart {
```

```
@ManagedComponent(      name        = "OrderService",
                        description = "The implementation of the order service" )

public class OrderImpl implements OrderService {

      @InjectComponent
      private ShoppingCart          shoppingCart;

      /**
       * Add a new article to the currently open order (
       * @param a
       */

      public void addArticleToShoppingCart( Artikel a ) {
            shoppingCart.addArticle( a );
      }
```

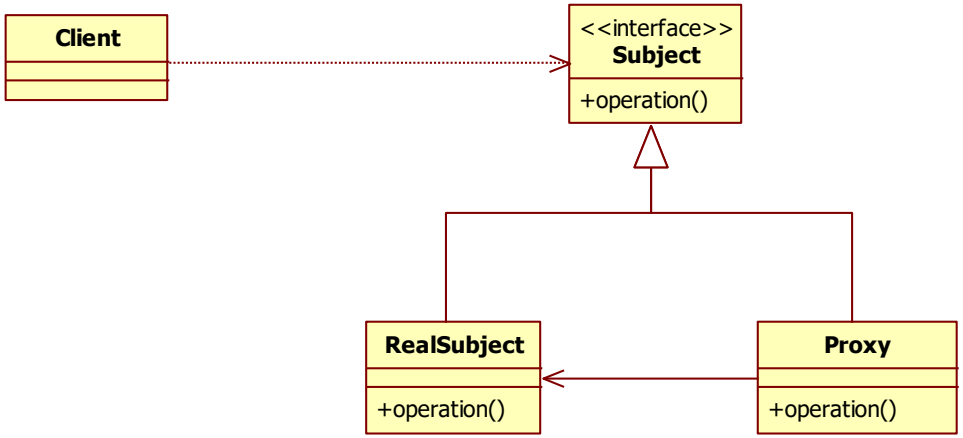
- Session-Handling wird in den Konfigurator verlagert

Lösung 3 – Eine Anwendung des Proxy-Musters

- **Zweck**:
  - Kontrolliere den Zugriff auf ein Objekt mit Hilfe eines vorgelagerten Stellvertreterobjektes
- **Beispiel:**
  - Lokaler Stub bei RMI
  - Partielles laden von Objekten

Lösung 3 – Eine Anwendung des Proxy-Musters

- **Lösung:**



Lösung 3 – Eine Anwendung des Proxy-Musters

Subject:

- Deklariert eine gemeinsame Schnittstelle von RealesSubjekt und Stellvertreter
- D.h. der Stellvertreter kann überall dort eingesetzt werden wo auch das RealeSubjekt engesetzt werden kann

Proxy:

- Verwaltet eine Referez auf das reale Objekt
- Kontrolliert den Zugriff auf das reale Objekt

RealSubject:

- Definiert das reale Objekt welches durch den Proxy geschützt werden soll

Lösung 3 – Eine Anwendung des Proxy-Musters

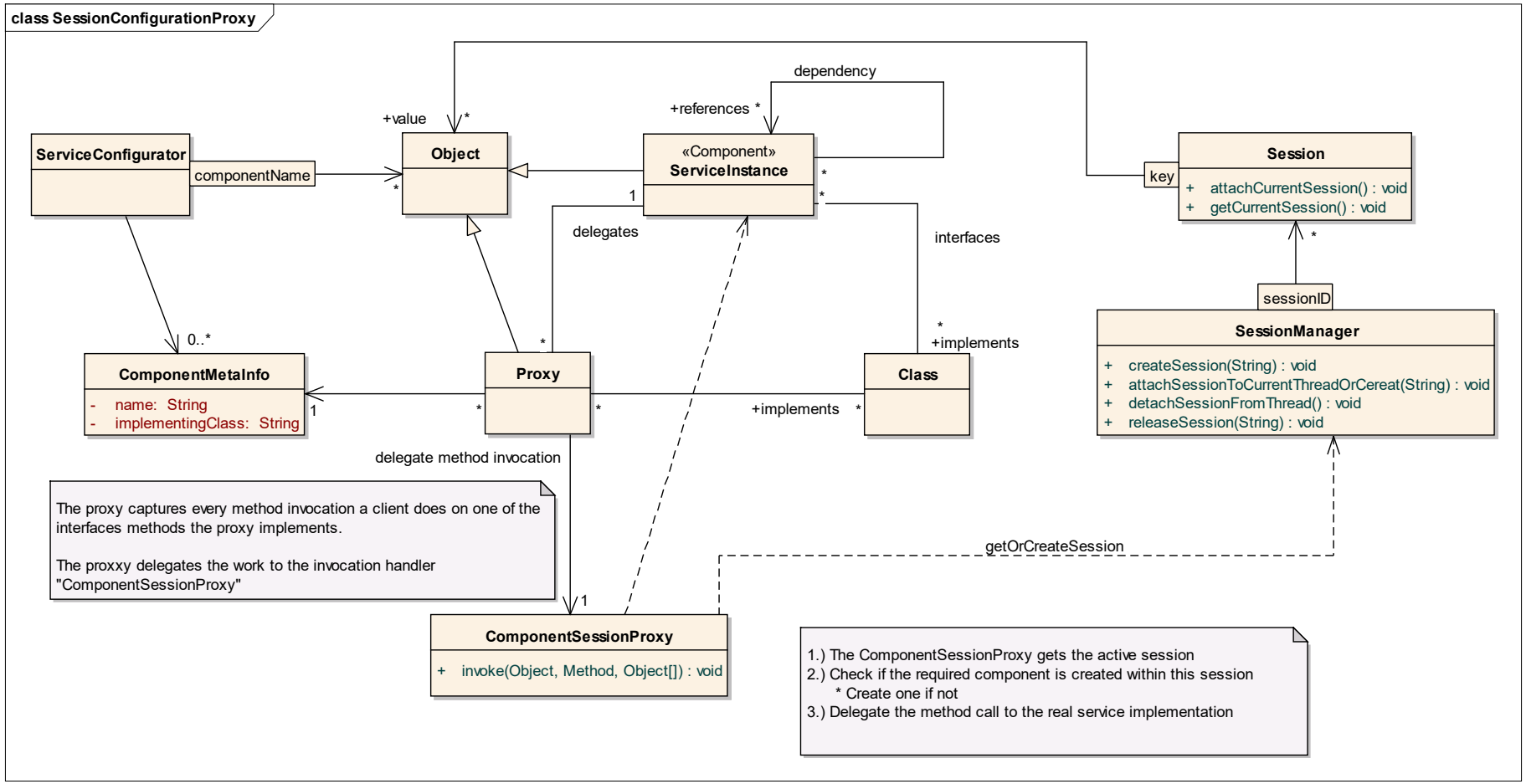
Bewertung:

Vorteile:

- Zugriff auf kostspielige Objekte ist für den Benutzer transparent

Nachteil:

- Da ein Proxy nicht identisch mit der Komponente selbst ist (als Objekt), muss man beim Testen auf Objekt-Identität vorsichtig sein.
- Ein Vergleich kann falsch ausgehen, obwohl dieselbe Komponente gemeint ist

Was bedeutet Session?
⇒ Fragen über Sessions sind wichtig.

Lösung 3 – Eine Anwendung des Proxy-Musters im Konfigurator